Lists and Indexing: Deep Dive

Why the implementation looks like this

Gregory S. DeLozier, PhD

The Node Is Called “complex”, Not “index”

```
node = {"tag": "complex", "base": node, "index": index}
```

Chapter 9 only ever produces this node from a `[index]` suffix. Nothing here uses the generality the name implies yet.

“complex” (continued)

Chapter 10 reuses this same mechanism for object field access by string key: the same `"complex"` node, just with a string index instead of an integer one. Chapter 11 adds function calls, syntactically the same form again: a base expression followed by a postfix suffix, `(args)` instead of `[index]`. The parser already builds a chain via `parse_complex_suffixes`; extending that loop to also accept `(` is a small addition to an existing function, not a redesign.

Naming the node for where it is going, not just what it currently does, is what makes both extensions small instead of a rename.

One Suffix-Parsing Function, Two Uses

```
target, tokens = parse_complex_suffixes(target, tokens[1:])
```

This is from `parse_assignment_statement`. It is the exact same function a read expression calls after any factor. An assignment target is not a different grammar production; it is the same node form, interpreted differently by the evaluator depending on which side of = it ends up on.

Target Before Value

Plain assignment has always evaluated the right-hand value, then resolved the target. For an identifier, resolving the target does nothing observable, so this order was invisible for eight chapters. Indexed assignment makes it observable, because resolving the target now means evaluating expressions.

Chapter 9 picks the opposite order: target first, value last. Uniformly, for every kind of target.

Target Before Value, Proof

`exit` cannot sit inside `a[exit(6)] = exit(999)` in real source: it cannot nest inside any expression, only stand alone as a statement. Built directly as AST nodes instead, the same proof holds:

```
# exit_node(n) builds the AST node for exit(n)
base = {"tag": "identifier", "value": "a"}
target = {"tag": "complex", "base": base, "index": exit_node(6)}
node = {"tag": "assign", "target": target, "expression": exit_node(999)}
evaluate(node, {"a": [1, 2, 3]})
```

Returns `(6, "exit")`. Not `(999, "exit")`.

The index is checked before the value is touched. Evaluated first, the value would win instead: `(999, "exit")`.

Chains Follow For Free

```
matrix_id = {"tag": "identifier", "value": "matrix"}
inner = {"tag": "complex", "base": matrix_id, "index": exit_node(7)}
zero = {"tag": "number", "value": 0}
target = {"tag": "complex", "base": inner, "index": zero}
node = {"tag": "assign", "target": target, "expression": exit_node(999)}
```

Returns `(7, "exit")`, from the *inner* index, before the outer index or the value is ever reached.

Index Typing

```
if type(index) is bool or type(index) not in (int, float):
    raise TypeError("Index must be an integer")
if type(index) is float and not index.is_integer():
    raise TypeError("Index must be an integer")
```

`3` and `3.0` are both valid indices. `3.5` is not. `true` is not, even though Python's own `isinstance(True, int)` is `True`.

No Negative Indices, No Silent Growth

```
index = int(index)
if not (0 <= index < len(base)):
    raise IndexError("Index out of range")
```

One bounds check, used for both reading and writing. `a[-1]` is out of range, the same as `a[99]`. `a[5] = 1` on a three-element list raises; it does not grow the list to make room.

One Store, Two Kinds of Destination

```
a = [1, 2, 3];
b = a;
b[0] = 99;
```

```
destination[key] = value
```

The new code resolves a list position. The store is unchanged.

Contrast: Equality Needed Real New Code

```
def vertex_equal(left, right):
    if type(left) is list and type(right) is list:
        if len(left) != len(right):
            return False
        return all(vertex_equal(l, r) for l, r in zip(left, right))
    numeric = type(left) in (int, float) and type(right) in (int, float)
    return (numeric or type(left) is type(right)) and left == right
```

Unlike aliasing, this could not be inherited. Python's own `[True] == [1]` is `True`.

length, slice, head, and tail Are Keywords, Not Calls

```
operations = {"number_conversion": "number", "string_conversion": "string",
              "boolean_conversion": "boolean", "type_query": "type",
              "length": "length", "head": "head", "tail": "tail"}
```

`length`, `head`, and `tail` join the same dict `number()`, `string()`, `boolean()`, and `type()` already used. Three new entries, not a new mechanism. `slice` takes three arguments, so it gets its own small parsing function.

Two Representations, One Function

```
def vertex_repr(value):
    if type(value) is bool:
        return "true" if value else "false"
    if type(value) is list:
        return "[" + ", ".join(vertex_repr(item) for item in value) + "]"
    if type(value) is str:
        escaped = ...
        return '"' + escaped + '"'
    return str(value)
```

A bare `print("hi")` still prints `hi`, unquoted, exactly as it always has. A string *inside* a list prints as `"hi"`, quoted.

Self-Reference Is Allowed, Not Rejected

```
a = [0];
a[0] = a;
```

Legal. `a[0]` is now the very list `a` names. Indexed assignment accepts any value, the same way plain assignment always has; nothing distinguishes “a list” from “this particular list” as a value, so nothing rejects this.

The Rejected Alternative

Reject `a[0] = a` at assignment. Checking would mean walking the entire right-hand value looking for a reference back to the list being assigned into, on every indexed assignment of a list value, forever, to guard against something that almost never happens.

What Was Chosen Instead

Let `==` and `print` tolerate the cycle. Comparing `a` to itself is the cheap case: identity short-circuits before any element is examined, matching how Python's own list `==` already behaves for `a==a`. Two *separately built* cycles are never identical, though, and Python's own list `==` has exactly that gap too: `a=[0]; a[0]=a; b=[0]; b[0]=b; a==b` raises `RecursionError` in plain Python, not just in Vertex.

The Real Cost

`vertex_equal` also tracks which pairs of lists are currently being compared, and `vertex_repr` tracks which lists are currently being printed; both add one set entry before recursing into a list and remove it after, an O(1) cost per list visited.

Not free, but flat. This adds the same small, constant bookkeeping to every list visited, cyclic or not, rather than growing with nesting depth the way copying the tracking set at every level would. Nothing is added to assignment either way.

What Carries Forward

The `"complex"` node is already built to extend. The `(value, status)` contract did not change. The only new category of decision this chapter introduced was ordering: deciding, for the first time, what “the target” means when it is not just a name. Chapter 10’s own keyed assignment inherits that same ordering question directly, by string key instead of position. Chapter 11 asks a related question for call arguments. This chapter is the reason both questions already have a precedent to follow instead of a blank page.